

Enterprise Employee Attrition Intelligence Platform — Technical Audit

Reviewed by: Senior DS / ML Eng / HR Analytics / Hiring Manager lens **Dataset:** IBM HR Analytics (1,470 rows, 35 cols, synthetic) **Models:** Logistic Regression, Decision Tree, Random Forest, XGBoost, LightGBM, CatBoost **Purpose of this document:** A fix-it list, ordered by severity, so you can repair the notebook before your next interview — not a confidence-destroying wall of text.

0. Bottom Line Up Front

This notebook is **visually the strongest attrition project most reviewers will have seen at student level** — 14 phases, SHAP, cost modeling, what-if simulation, executive dashboards. That breadth is real and will catch a recruiter's eye scrolling GitHub.

But there is **one Critical bug that actively produces wrong numbers in two places**, and **one Critical methodological gap (no leakage-proofed pipeline, no nested CV)** that any ML engineer interviewer will find within 90 seconds of asking "walk me through your train/test split." Both are fixable in under an hour. Everything else is High/Medium polish.

Verdict if shipped as-is to a FAANG/Big 4 interviewer: "ambitious but the fundamentals weren't checked." **Verdict after the Critical + High fixes below:** "strong portfolio project, borderline industry-level."

1. CRITICAL Issues (fix before any interview)

1.1 — The headline ROI/savings number is fabricated by a unit mismatch

Where: Phase 9, the `(comps)` dict and the final `(print(f"FINANCIAL INSIGHT: Investing ${total_cost*0.12/1e6:.2f}M..."))` line.

The bug: `(total_cost)` is computed from `(replacement_cost())`, a function with its own internal cost structure. The "Investing \$X to save \$Y" line then multiplies `(total_cost * 0.12)` — an arbitrary 12% retention-budget assumption that **appears nowhere in the cost model, is never explained, and isn't derived from any of the four cost components shown in the breakdown chart** (Recruitment/Training/Productivity/Knowledge). The breakdown chart itself separately recomputes those four components from scratch using *different* formulas (`(Annual_Salary*0.30)` for recruitment vs. the `(recruit_pct)` tiered logic used in `(replacement_cost())`) — so the bar chart and the headline number don't even agree with each other internally.

Why it matters: This is the **one number every CHRO in the room will ask about** ("how did you get 5x ROI?"). Right now the honest answer is "I picked 12% and 60% retention rate without justification, and my cost breakdown chart uses different math than my cost total."

That's the fastest way to lose credibility with an HR Director persona — they live and die by defensible cost models (SHRM/Gallup benchmarks exist specifically so you don't have to invent percentages).

Fix:

```
python

# Make every assumption a named, justified constant — and use ONE formula every
RETENTION_PROGRAM_COST_PCT = 0.12 # cite: typical retention program spend as
EXPECTED_RETENTION_SUCCESS_RATE = 0.60 # cite: literature range is 20-50%; jus

retention_program_cost = at_risk["Annual_Salary"].sum() * RETENTION_PROGRAM_COST_PCT
savings = total_cost * EXPECTED_RETENTION_SUCCESS_RATE
roi = savings / retention_program_cost

# And rebuild the cost breakdown chart by calling replacement_cost() and decompose
# ITS internal terms, not a second, different formula:
def replacement_cost_breakdown(monthly_income, job_level, years_at_company):
    annual = monthly_income * 12
    recruit_pct = 0.20 if job_level <= 2 else (0.35 if job_level <= 3 else 0.50)
    recruitment = annual * recruit_pct
    training = annual * 0.10
    ramp_months = min(3 + job_level, 6)
    productivity = monthly_income * ramp_months * 0.50
    knowledge = annual * min(years_at_company * 0.02, 0.20)
    return recruitment, training, productivity, knowledge
```

Also state the 60% retention-success assumption as a **sensitivity range** (e.g., show ROI at 30%/45%/60%) rather than a single point estimate — that alone reads as far more senior.

Severity: CRITICAL. This isn't a style nit — the chart and the headline contradict each other, and an interviewer who notices will discount everything else in Phase 9 and 11.

1.2 — Scaler and SMOTE are fit in a way that risks subtle leakage into the "all employees" risk scores

Where: Phase 7, `(X_all_sc = scaler.transform(X))` then `(risk_proba = best_model_obj.predict_proba(X_all_sc))`.

The bug: This itself is *technically* fine (transform, not fit_transform — good). But look at what `(X_all_sc)` actually is: it's **the full 1,470-row dataset, including the 1,176 rows the model was trained on**. The Phase 7 "Risk Engine" then presents Risk Scores for the entire workforce, mixing in-sample predictions (overfit, artificially confident) with true out-of-sample predictions (the 294 test rows) **with no distinction shown anywhere in the UI/report**.

Why it matters: Every downstream number that touches `risk_df` — the cost model in Phase 9, the what-if simulator in Phase 10, the executive dashboard in Phase 11, the heatmaps in Phase 12 — is built on probabilities that are **partially in-sample**. For the 1,176 training-set employees, the model has already seen their label during training, so `predict_proba` on them is optimistically biased (this is exactly what train/test splits exist to prevent, and the notebook correctly does it for evaluation in Phase 5 — then quietly throws that discipline away in Phase 7 onward).

Why an interviewer will catch this: "Your evaluation says 81% ROC-AUC, but your risk engine assigns scores to the same people the model trained on — so is the 11% Critical Risk number real, or partly the model just recognizing rows it memorized?" There's no good answer to that as the notebook stands.

Fix — two acceptable options:

```
python
```

```
# OPTION A (recommended): Only ever score held-out data in production-facing ou
# Refit final model on ALL data only after you've reported test-set metrics hon
# and clearly label that Phase 7+ uses a "production model" distinct from the "

# OPTION B (if you must score the full historical set, e.g. for a real HR tool)
# Generate cross-validated out-of-fold predictions for the training rows so EVE
# row's score is genuinely out-of-sample:
from sklearn.model_selection import cross_val_predict
oof_proba = cross_val_predict(best_model_obj, X_train_sc, y_train, cv=skf, meth
test_proba = best_model_obj.predict_proba(X_test_sc)[: ,1]
# stitch oof_proba (train rows) + test_proba (test rows) back into risk_df by o
```

At minimum, **add a column** `Data_Split` (`"Train"`/`"Test"`) to `risk_df` and a one-line caveat in the Phase 7 markdown: *"Risk scores for training-set employees may be optimistically biased; only test-set scores (n=294) are validated out-of-sample."* That single sentence demonstrates the statistical maturity a Senior DS interviewer is screening for.

Severity: CRITICAL for the "is this production-ready / would a company trust it" axis — not because the code crashes, but because it silently blends validated and unvalidated predictions in the artifact executives would actually read.

1.3 — No nested CV / no held-out validation set for hyperparameter selection

Where: Phase 4.3–4.4, every `GridSearchCV`/`RandomizedSearchCV` call.

The bug: Hyperparameters are tuned using 5-fold CV scored on `roc_auc`, and the **same test set** (`X_test_sc`) is then used in Phase 5 to report final performance. This is single train/test/no-nested-CV — the test set was never touched during tuning here, which is good, but there is **no validation set distinct from both** to catch the (very real, given

n=1,470) risk that the "best" hyperparameters were selected partly by CV-fold noise rather than a generalizable signal. With only 1,176 training rows split 5 ways, each fold is ~235 rows — variance in fold-level AUC will be substantial, and `RandomizedSearchCV` with `n_iter=15` over a sizeable grid is searching a lot of configurations against a noisy signal.

Why it matters: This is the single question every ML Engineer interviewer asks about any tuned model: *"how do you know your test-set AUC isn't partly luck from which 294 rows ended up in the test split?"* Right now there's no repeated-CV or multiple-seed check to answer that.

Fix (cheapest credible version — do this one):

```
python

# Repeat the train/test split + full pipeline across multiple seeds and report
from numpy import mean, std
seeds = [42, 7, 123, 2024, 99]
auc_scores = []
for s in seeds:
    Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.20, random_state=s,
    sc = StandardScaler().fit(Xtr)
    Xtr_s, Xte_s = sc.transform(Xtr), sc.transform(Xte)
    Xtr_sm, ytr_sm = SMOTE(random_state=s).fit_resample(Xtr_s, ytr)
    m = XGBClassifier(**model_xgb.get_params()) # reuse tuned hyperparams, ref
    m.fit(Xtr_sm, ytr_sm)
    auc_scores.append(roc_auc_score(yte, m.predict_proba(Xte_s)[:,:1]))
print(f"ROC-AUC across 5 seeds: {mean(auc_scores):.4f} ± {std(auc_scores):.4f}")
```

Report that $\pm$ in the leaderboard, not just a single point estimate. This single addition is the highest-leverage 20 minutes of work in the whole notebook for an ML Engineer interview.

Severity: CRITICAL for ML rigor scoring specifically (not for "does it run").

2. HIGH Severity Issues

2.1 — `Tenure_Group` and `Age_Band` leak `YearsAtCompany`/`Age` derivatives into the model inconsistently

Where: Phase 4.1: `df_model.drop(columns=`
`["Attrition_Num", "Education_Label", "Age_Band", "Tenure_Group"],`
`errors="ignore")`.

Issue: `Tenure_Group` (a *categorical bucketing* of `YearsAtCompany`) is engineered in Phase 3 specifically as a model-ready feature, displayed with attrition-rate charts implying predictive value — then **dropped before modeling** in Phase 4.1. Meanwhile its continuous parent (`YearsAtCompany`) stays in. So Phase 3 markdown promises "6 composite features"

feeding the model, but one of the six (`Tenure_Group`) never reaches (`X`). This is either a half-finished feature or undocumented dead code — either way, it's inconsistent with the notebook's own claims ("Engineer 6 composite features... directly into the model").

Why it matters: A technical interviewer reading the feature engineering section, then checking (`X.columns`), will notice the count doesn't match the narrative. Small, but it's exactly the kind of "did they actually check their own work" signal that separates a 7/10 from a 9/10 portfolio piece.

Fix: Either one-hot encode (`Tenure_Group`) and keep it in (`X`) (and check correlation with (`YearsAtCompany`) first — see 3.3), or remove the "6 composite features" claim from the Phase 3 markdown and explicitly state which are used for modeling vs. which are for HR-facing visualization only. The second is actually the more defensible answer in an interview ("this feature was for stakeholder communication, not the model — here's why") — just say that explicitly in the notebook.

2.2 — SMOTE/SMOTETomek resampling, then evaluating on the original (unscaled-class-ratio) test set, is correct — but it's never explained, and the choice between SMOTE vs SMOTETomek per-model is unmotivated

Where: Phase 4.2-4.4. LR/DT/RF trained on (`X_tr_sm`) (SMOTE); XGB also (`X_tr_sm`); LightGBM/CatBoost trained on (`X_tr_stk`) (SMOTETomek).

Issue: The actual mechanics here are correct — resampling only the training fold, evaluating on the untouched, real-distribution test set is exactly right and avoids a classic leakage mistake. Good instinct. The problem is **why do three models get SMOTE and two get SMOTETomek, with no ablation showing that choice was deliberate?** Right now it reads as "tried both, used whichever was on hand" — which an interviewer will ask about, and "no specific reason" is a weak answer.

Fix: Either (a) run every model with both resampling strategies and report which wins per-model in the leaderboard (turns this into a legitimate ablation — strong signal), or (b) standardize on one strategy for all six models and drop the unused import. Given XGBoost/LightGBM/CatBoost all have native (`scale_pos_weight`)/(`class_weight`) handling, also worth comparing **SMOTE vs. class-weighting** — that comparison alone is something a Senior DS would specifically ask about for a 16.1%-imbalance problem, since SMOTE on tree-based gradient boosters is debatable value-add vs. just reweighting.

2.3 — Model selection is by ROC-AUC alone, but the business problem (Phase 5's own framing) wants Recall/Capture Rate prioritized

Where: Phase 5: (`leaderboard.sort_values("ROC AUC", ascending=False)`) → XGBoost selected as (`best_model_name`).

Issue: The notebook's own markdown says: *"Attrition Capture Rate (Recall) tells HR what fraction of actual leavers the model identifies."* That's the right business framing — missing a leaver (false negative) is far more costly to HR than over-flagging a stayer (false positive), because the entire point of the tool is *intervention before someone quits*. Yet the model actually selected, XGBoost, has **Capture Rate 38.3%** — it misses 6 of every 10 people who actually leave — while Logistic Regression, which the notebook itself trained and evaluated, has **Capture Rate 59.6%**, nearly 1.6x better at the one metric the business case is built on.

Why it matters: This is the single most consequential modeling decision in the notebook and it's optimized for the wrong objective given the notebook's own stated goals. A Hiring Manager or HR Director persona will ask: "you told me Capture Rate is what matters to HR — why did you pick the model with the worst Capture Rate of your top performers?" There's no defensible answer in the current structure.

Fix:

```
python

# Pick the selection metric based on the cost asymmetry, not generic AUC.
# If a missed leaver costs ~$38.6K (your own Phase 9 number) and a false-positi
# intervention costs maybe $500-2,000 (a check-in, mentor assignment), the opti
# threshold/model favors Recall heavily. At minimum, make best_model_name selec
SELECTION_METRIC = "Recall" # or build a custom cost-weighted score:
def business_score(r, fn_cost=38597, fp_cost=1000):
    cm_cost = r["_fn"]*fn_cost + r["_fp"]*fp_cost # requires storing FN/FP co
    return -cm_cost
leaderboard["Business Cost"] = [business_score(r) for r in eval_results]
best_model_name = leaderboard.sort_values("Business Cost").iloc[0]["Model"]
```

Alternatively, keep XGBoost for its AUC/ranking quality but **lower its decision threshold** below the default 0.5 to trade precision for recall, and justify the new threshold using the cost asymmetry from Phase 9. Either fix turns a glaring inconsistency into one of the strongest "I understand the business" answers in the whole interview.

2.4 — `use_label_encoder` passed to `XGBClassifier` (deprecated/removed parameter)

Where: Phase 4.4: `XGBClassifier(random_state=42, eval_metric="logloss", use_label_encoder=...)` (cut off in PDF, but present).

Issue: `use_label_encoder` was deprecated in XGBoost 1.x and **removed entirely in XGBoost ≥2.0** — passing it now either silently warns or raises a `TypeError` depending on installed version, which is exactly the kind of "works on my Colab, breaks in interview/CI" landmine. Given the notebook installs unpinned (`!pip install -q lightgbm catboost imbalanced-learn shap`) with no version pins, and `xgboost` isn't even in that install line —

it must be pre-installed in Colab at whatever version Colab ships), this is a real reproducibility risk.

Fix:

```
python

# Remove use_label_encoder entirely – not needed in modern XGBoost
model_xgb_search = XGBClassifier(random_state=42, eval_metric="logloss")
```

And pin versions in the setup cell:

```
python

!pip install -q "lightgbm==4.3.0" "catboost==1.2.5" "imbalanced-learn==0.12.3"
```

2.5 — SHAP `TreeExplainer` + `shap_values[1]` pattern is version-fragile and will break silently on newer SHAP

Where: Phase 6.1: `shap_values = explainer.shap_values(X_test_df); shap_vals = shap_values[1] if isinstance(shap_values, list) else shap_values`.

Issue: This pattern (list-indexing `[1]` for the positive class) is the **old SHAP API** for multi-output models. In SHAP ≥ 0.42 , `TreeExplainer.shap_values()` on binary XGBoost/LightGBM/CatBoost classifiers returns a **single 2D array**, not a list of two arrays — so the `isinstance(..., list)` branch is dead code on a modern install, and the values being plotted are correct only by coincidence of how the code happens to fall through. For CatBoost specifically, output shape conventions have changed across versions and this exact pattern is a known source of "SHAP plot looks right but is plotting the wrong class" bugs.

Fix:

```
python

import shap
explainer = shap.TreeExplainer(best_model_obj)
explanation = explainer(X_test_df) # new API – always returns shap.Explanation
# explanation.values shape: (n_samples, n_features) for binary, or (n_samples,
shap_vals = explanation.values if explanation.values.ndim == 2 else explanation
shap.summary_plot(shap_vals, X_test_df, plot_type="bar", max_display=20)
```

This is worth fixing even though the current output *looks* fine in this run — it's the kind of bug that resurfaces the moment someone reruns the notebook on a fresh Colab instance six months from now (a very likely interview scenario: "can you run this live for me?").

2.6 — Engagement_Index and the four SAT_COLS it's derived from are both fed into the model simultaneously

Where: Phase 3 feature engineering (`Engagement_Index`) = mean of normalized (`JobSatisfaction`), (`EnvironmentSatisfaction`), (`RelationshipSatisfaction`), (`WorkLifeBalance`) and Phase 4.1 (none of the four source columns are dropped before modeling).

Issue: This isn't leakage, but it is **redundant collinear input** — the model sees both the raw satisfaction scores and a near-deterministic linear combination of them. For tree-based models this mostly just dilutes feature importance (SHAP correctly shows `JobSatisfaction` and `Engagement_Index` both ranking, splitting credit that should arguably consolidate), and for Logistic Regression it can inflate coefficient variance. The SHAP beeswarm in Phase 6.2 even shows this: `Engagement_Index` and `EnvironmentSatisfaction`/`JobSatisfaction` show overlapping, partially redundant signal.

Fix: Either drop the four `SAT_COLS` once `Engagement_Index` is built (cleaner, and tests whether the composite *actually* carries the predictive signal on its own — a more interesting result to report), or keep both but explicitly call out the multicollinearity in the Feature Engineering Audit section of your write-up and show a correlation matrix to prove you checked. Right now it looks unexamined.

3. MEDIUM Severity Issues

3.1 — Data Quality Score formula is ad hoc and not validated

Where: Phase 1: `score = 100 - missing_pct*2 - min(outlier_pct*0.5, 10); if is_constant: score -= 30; if is_id: score -= 20`.

These weights (`*2`), (`*0.5`), (`-30`), (`-20`), cap of (`10`) appear to be invented rather than derived from any cited methodology. For a project marketed as "Enterprise" and "production-grade," a Data Analyst interviewer will ask where these coefficients came from. Either cite a real DQ framework (e.g., DAMA-DMBOK dimensions) or relabel this as a heuristic illustrative score, not an authoritative metric — and say so in the markdown.

3.2 — IQR-based outlier detection applied to inherently bounded/ordinal columns

Where: `compute_dq_report`, the outlier block applied uniformly to every `int64`/`float64` column.

This includes columns like `PerformanceRating` (2 unique values), `JobLevel` (1-5 ordinal), `Education` (1-5 ordinal) — IQR-based outlier flagging is meaningless on near-binary or 5-point ordinal scales, and the Phase 1 output table itself shows this going wrong: `PerformanceRating` flagged at **15.37% outliers** and `TrainingTimesLastYear` at **16.19%** — these aren't outliers, they're just the natural shape of a discrete/ordinal distribution. This

inflated outlier percentage then *drags down the Data Quality Score* for those columns for no statistically valid reason.

Fix: Branch outlier detection — skip IQR for columns with `nunique() <= 10` (treat as categorical/ordinal) or use a method appropriate to discrete data (e.g., frequency-based rarity rather than IQR).

3.3 — No multicollinearity check (VIF or correlation matrix) anywhere in the notebook

For Logistic Regression specifically, with 49 features including several engineered composites built from other engineered composites (`Flight_Risk_Score` is built from `Promotion_Gap`, `Engagement_Index`, `Burnout_Score`, and `Compensation_Ratio` — all four of which are *also* independently in `X`), this is a real gap. A correlation heatmap of the final feature matrix (10 minutes of work) would be a high-value addition and is something a Data Scientist interviewer will specifically ask "did you check for multicollinearity?" about, given how the feature engineering is structured.

3.4 — `Flight_Risk_Score` is included as both an engineered *feature* and conceptually competes with the model itself

Where: Phase 3 markdown calls it "Pre-model composite risk metric," but it's then fed directly into `X` in Phase 4 and shows up in SHAP importance in Phase 6 (rank ~6).

This is conceptually muddled: if `Flight_Risk_Score` is a **hand-built heuristic risk score**, and the **ML model's whole purpose is to learn a better risk score from data**, then including the heuristic as an input to the model that's supposed to replace it is circular — the model partially just learns to trust your hand-tuned formula. It's not leakage (no future information), but it undermines the "ML model outperforms naive heuristics" narrative the rest of the project is implicitly selling. Worth adding one analysis: **compare `Flight_Risk_Score` alone as a baseline classifier (AUC) against your trained models** — if the gap is small, that's an important, honest finding to report, not hide.

3.5 — `display(df.head(3))`, `display(dq_report...)` calls assume Colab; no graceful fallback / no `.ipynb` reproducibility check

The notebook is fully Colab-coupled (`google.colab` import, IPython `display()` used throughout without an `IPython.display` import guard for non-notebook execution). Fine for a Colab artifact, but if this is meant to demonstrate "production-readiness" (Phase 13 explicitly claims this), it should at minimum note that Phases 1-12 are exploratory/notebook-only and only Phase 13/14's artifacts (`joblib`, `json`) are deployment-relevant. Right now the notebook conflates "I drew an architecture diagram" with "this is production-ready," which a hiring manager will see through immediately — see Section 7.

3.6 — Hardcoded dataset URL with no checksum/version pin

`DATASET_URL` points to a third-party GitHub raw file with no version pinning, hash check, or local fallback. If that repo is renamed, deleted, or the file changes, the notebook silently

breaks or silently produces different numbers on rerun. For a portfolio piece meant to be rerun by a recruiter, add a fallback: try the URL, except → load a bundled local CSV copy.

4. LOW Severity / Polish

- **Phase 1 dashboard** (`axes[1]`) ("Top Columns by Outlier %") uses (`nlargest`) on a dtype-filtered frame but the figure shows ordinal/binary columns dominating (see 3.2) — once 3.2 is fixed this chart becomes genuinely useful instead of misleading.
 - **PALETTE list is truncated in several plt calls** (`color=[ "#e94560", "#f5a623", "#2ecc71" ][:len(dept_rate)]`) — fragile if (`Department`) ever has >3 categories; index by name via a dict instead of positional slicing.
 - (`shap.initjs()`) is called but no interactive force-plots are actually used anywhere in the notebook — dead code, remove it or use it (a force plot for the top-risk employee would be a nice addition to Phase 6.4).
 - Repeated (`plt.tight_layout(); plt.show()`) pattern is fine but every figure recomputes (`groupby`) aggregations that are reused across cells (e.g., (`dept_rate`) in Phase 2.2 vs (`dept_risk`) in Phase 7 vs (`dept_summary`) in Phase 11 vs (`dept_cost`) in Phase 9 — four separate, slightly-differently-named department-level aggregations of overlapping data). Consolidate into one (`department_summary`) table computed once, referenced everywhere. This is both a performance win (minor, n=1,470 is tiny) and, more importantly, a **maintainability/DRY signal** that matters far more for an ML Engineer interview than raw speed at this data size.
 - (**KPI**) Plotly indicators (Phase 11) use (`mode="number", value=0`) with the actual value baked into (`title.text`) HTML — a hack that works visually but means the Plotly object's actual data is meaningless (`value=0`) for every card). Use (`mode="number", value=<actual numeric value>, number={"prefix":...}`) properly so the artifact isn't relying on string formatting tricks inside a title field.
-

5. Performance Review

At **n=1,470 rows**, no operation in this notebook is meaningfully slow in absolute terms — this is not a performance-bound notebook, and claiming "performance optimization" matters much here would be artificial. That said, for the stated "production-grade" framing:

Pattern	Where	Issue	Fix
Repeated <code>df.groupby(...)</code> on the same keys across 4+ phases	Phases 2, 7, 9, 11	Recomputed from scratch each time	Compute once into a shared <code>department_summary</code> table (also fixes 4.x DRY issue above)
<code>RandomizedSearchCV(..., n_jobs=-1)</code> nested with <code>StratifiedKFold</code> on a 1,176-row training set	Phase 4.4	Parallelism overhead likely exceeds benefit at this scale; fine, just not meaningfully "optimized"	Not worth fixing — correctly notes priority should go to nested-CV correctness (1.3) over speed
<code>risk_df = df_fe.copy()</code> then re-deriving most of <code>df_fe</code> 's columns again	Phase 7	Unnecessary full-frame copy when only a few new columns are added	<code>risk_df = df_fe.assign(Risk_Score=..., Risk_Proba=..., Risk_Category=...)</code> avoids the copy

Realistic verdict: there is no performance crisis here. Don't over-invest in "optimization" framing for a 1,470-row dataset — an interviewer will see through manufactured performance claims on data this size faster than they'll appreciate them. The DRY/consolidation point is worth doing for code-quality signal, not speed.

6. Visualization Audit

Strong: The demographic, satisfaction, and SHAP visualizations are genuinely well-chosen chart types (boxplots for income-by-level, violin plots with `split=True` for distribution comparison, beeswarm for SHAP — these are the *correct* advanced chart choices, not just defaults). The waterfall plot for individual-employee explanation is a strong, recruiter-visible differentiator most student projects skip entirely.

Needs work:

- Too many 2×3 and 1×3 subplot grids back-to-back** (Phases 2.1, 2.2, 2.3, 3) — by Phase 3 a reader has seen ~17 charts before reaching a single model result. For an executive audience this is fatigue-inducing; consolidate Phase 2 into fewer, denser dashboards or move detail charts to an appendix and lead with 3-4 headline visuals.
- The Department Risk & Cost Impact table** (the screenshot you attached) is a good example of dense, useful tabular reporting — but the `Avg_Risk` color-gradient (red=Sales 22.0, green=R&D 16.0) uses a continuous gradient across only 3 rows,

which is statistically misleading: a 3-point range (16.0–22.0) is being displayed with the same visual intensity range as if it spanned 0–100. Rescale gradients to the actual risk-score range (0–100) so color intensity is comparable across tables, not just within one table.

- **Plotly dark theme** (`template="plotly_dark"`) **mixed with matplotlib light-background charts** throughout — visually inconsistent if someone screenshots the notebook for a deck (which is exactly what an "Executive Dashboard" phase should be optimized for). Pick one visual language.
 - **KPI cards (Phase 11)** render fine in Colab but the `go.Indicator` hack noted in 4.x means they won't survive being exported to a static image cleanly — if the goal is "send this to a CHRO," confirm it actually screenshots/exports correctly.
-

7. Business Value Audit (CHRO / HR Director lens)

What's genuinely good: The Overtime $\rightarrow$ 2.9x attrition multiplier finding, the Department $\times$ Salary Band heatmap, and the Tenure $\times$ Satisfaction heatmap are the kind of finding an actual HR Director would act on immediately — overtime policy and early-career compensation are concrete, fundable interventions. The retention recommendation engine (Phase 8) producing per-employee, prioritized actions is the right *shape* of deliverable for HR (not just a risk score with no next step).

What's missing for real adoption:

1. **No manager-level rollup.** Everything is individual-employee or department-level. HR Directors act through managers — a "which of my direct reports are at risk, and why" view per manager is the single most-requested feature in real attrition tools and is entirely absent here.
2. **No time dimension.** This is a single snapshot. Real HR tools track *risk trend* (is this person's score rising over the last 3 reviews?) — without a time axis, "Critical Risk: 162 employees" is a number with no trajectory, and HR can't tell if an intervention from last quarter worked.
3. **Retention recommendations aren't grounded in cost-effectiveness.** Phase 8's rule engine assigns "+12% retention probability" to Workload Redistribution and similar figures — **these percentages are not derived from anything in the notebook** (no citation, no backtest, no A/B reference). An HR Director will ask "how do you know it's +12% and not +3%?" and currently there's no answer. Either cite literature (there is real published research on intervention effectiveness, e.g., Gallup's engagement-intervention studies) or relabel these as "directional priority," not a percentage.
4. **No fairness/adverse-impact check.** A model that ranks employees by attrition risk and feeds HR intervention decisions is exactly the kind of model that should be checked for disparate impact across gender, age, and marital status (all of which are *in* the feature set, and `MaritalStatus_Single` shows up in the SHAP top-20). This is not just an academic concern — in the US, using protected-class-correlated features to

drive employee-facing HR decisions has real EEOC exposure. At minimum, add a fairness audit: compare false-negative rates across gender and age bands. For an "Enterprise Platform" claim, this is a meaningful gap, not a nice-to-have.

5. **Cost model is plausible but not auditable** (see 1.1) — fix that first, since it's the number executives will anchor on.

8. Production Readiness Audit

Dimension	Status	Notes
Reproducibility	Partial	<code>np.random.seed(42)</code> set, but unpinned package versions (2.4) undermine this
Modularity	Weak	Single linear notebook; no functions extracted into an importable module; re-running Phase 7 alone is impossible without running 1-6 first
Maintainability	Weak	Four near-duplicate department aggregations (4.x); magic numbers throughout (1.1, 3.1)
Error handling	Minimal	One <code>try/except</code> around SHAP explainer selection; nothing else has any error handling — a malformed input row, a missing column, or a schema drift would crash the entire pipeline with no graceful degradation
Logging	Absent	<code>print()</code> statements only; no structured logging, despite Phase 13's architecture diagram explicitly specifying "Structured JSON... Every prediction"
Configuration management	Absent	No config file/class; thresholds (20/45/70 risk bands, 0.45 engagement cutoff, etc.) are hardcoded inline across multiple cells
Model persistence	Present	<code>joblib.dump</code> + JSON schema — this part is actually done correctly
Security	Not addressed	No input validation, no consideration of PII handling for what is explicitly HR/employee data (names aren't in this dataset, but a real deployment would need to address this)

Production-Readiness Score: 38/100.

This is not a harsh score given the framing — it's an honest one. Phase 13's architecture diagram is a *design document*, not implemented infrastructure, and that's fine **if labeled as such**. The risk is presenting Phase 13 as if it describes what Phases 1-12 already do.

Recommend retitling Phase 13 to "Proposed Production Architecture (Not Implemented)" — that single honest label converts a potential credibility problem into a demonstration of architectural thinking, which is exactly what you want a senior interviewer to see.

9. Scores

Lens	Score	Why
Recruiter Score	8/10	Visually impressive, broad scope, will get you a callback
Data Scientist Score	6/10	Good EDA instincts, but model selection logic (2.3) and validation rigor (1.3) are the kind of gaps a DS interview is specifically designed to find
ML Engineer Score	5/10	No nested CV, version-fragile SHAP code, in-sample/out-of-sample blending (1.2) are all things an ML Eng interview probes directly
Business/HR Analyst Score	7/10	Strong instincts (overtime, salary-band heatmaps) undercut by an unaudited cost model (1.1, 7.3)
Overall Portfolio Score	6.5/10 as-is → 8.5/10 after Critical+High fixes	The gap between these two numbers is entirely closeable in a focused 1-2 day pass

10. Final Verdict

Strengths

- Breadth and ambition: 14 phases covering the full lifecycle (EDA → modeling → XAI → cost → simulation → dashboard) is rare at this level
- Correct *mechanics* of train/test isolation for scaling and SMOTE (genuinely easy to get wrong, done right here)
- SHAP global + local + waterfall implementation is more complete than most professional internal tools
- Visualization chart-type choices are advanced and largely appropriate, not default/lazy
- Business framing (Capture Rate / Intervention Accuracy naming) shows real translation instinct between ML and HR language — even though the model selection didn't follow through on it (2.3)

Weaknesses

- Headline financial ROI number doesn't reconcile with its own supporting chart (1.1)
- Risk scores mix in-sample and out-of-sample predictions with no disclosure (1.2)
- No validation-rigor signal (repeated CV, nested CV) anywhere (1.3)
- Model selection metric contradicts the project's own stated business priority (2.3)
- Several "enterprise" claims (production-readiness, retention-probability percentages) aren't substantiated and would not survive a skeptical follow-up question

Must Fix Before Submission (Critical only)

1. Reconcile the Phase 9 cost/ROI numbers — one formula, used consistently, with named/justified constants (1.1)
2. Label or fix the in-sample/out-of-sample blending in the Risk Engine (1.2)
3. Add repeated-CV or seed-stability reporting for at least your top 2 models (1.3)

Nice-to-Have Improvements

All of Section 2 (High) and Section 3 (Medium) — none are launch-blockers for an interview, but each is a specific, likely follow-up question.

Is This Project Ready For:

Use case	Ready?
GitHub Portfolio	Yes — already a strong visual entry, fix 1.1-1.3 first
Internship Applications	Yes , comfortably
Data Analyst Roles	Yes
Business Analyst Roles	Yes , once Section 7 gaps (fairness, manager rollup) are at least mentioned as "future work"
Data Scientist Roles	Conditional — yes after fixing 1.1-1.3 and 2.3; not yet as-is
ML Engineer Roles	Conditional — yes after 1.2, 1.3, 2.4, 2.5; the validation-rigor gap is the one ML Eng interviewers will find fastest
Enterprise Demonstration	No — not until Section 8's gaps are either fixed or explicitly scoped as "design proposal, not implementation"

Overall Assessment

Strong Portfolio Project (current state) → **Industry-Level Project** (after the 3 Critical + 5

High fixes above, realistically 1-2 focused days of work)

The distance to "Enterprise-Grade" is larger and not worth chasing for interview purposes — that tier requires actual deployed infrastructure, monitoring, and fairness auditing that go beyond what a notebook-based portfolio piece should attempt. Aim for "Industry-Level" as the target, defend it well, and you're in strong shape.